

Design Movie Ticket Booking

Salesforce · Microsoft · Uber · Amazon · Swiggy – ఇదే నమూనా flight booking, hotel, restaurant table, event ticket అన్నిటికీ వర్తిస్తుంది.

ఇక్కడ మూడు విరుపులు ఉన్నాయి, మరియు మొదటిది ఒక **modelling** తప్పు – చాలా మంది దాన్ని గమనించకుండా ముందుకు వెళ్ళి, తర్వాత ఇరుక్కుపోతారు. రెండోది Deep Dive 01 యొక్క race, ఇక్కడ **డబ్బుతో**. మూడోది ఈ problem యొక్క నిజమైన గుండె: **payment** ఖాళీ.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output నిజంగా `node` lo run చేసినదే.

ఈ **PROBLEM** ఒక్కటి నేర్చుకుంటే – నాలుగు వస్తాయి

Movie booking, flight seat, hotel room, restaurant table, concert ticket – ఇవన్నీ **ఒకే problem**. ఒక పరిమిత inventory, ఒక సమయ పరిధి, పలు users ఒకేసారి, మరియు మధ్యలో ఒక **payment**.

అందుకే ఈ doc lo నేర్చుకున్నది – ముఖ్యంగా §11 lo ఉన్న **hold** నమూనా – మిగతా అన్నిటికీ నేరుగా వర్తిస్తుంది. Interview lo "Design BookMyShow" అడిగినా, "Design Airbnb booking" అడిగినా, మీ జవాబు యొక్క కేంద్రం ఒకటే.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఒక్క వాక్యం – లోపల మూడు సమస్యలు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఒక కుర్చీ, రెండు shows

3. Step – అతి సహజమైన మొదటి ప్రయత్నం
4. మొదటి విరుపు – 6pm కి book చేస్తే 9pm కూడా పోయింది
5. Step – భౌతిక వస్తువుని, show స్థితిని విడదీయడం

Part 3 – రెండో విరుపు: ఇద్దరికీ ఒక కుర్చీ

6. రెండో విరుపు – డబ్బుతో కూడిన race
7. Step – Atomic claim, మరియు కొత్త కష్టం: అన్నీ లేదా ఏమీ లేదు

Part 4 – మూడో విరుపు: payment ఖాళీ

8. మూడో విరుపు – రెండు దారులు, రెండూ తప్పు
9. Step – మూడో స్థితి: HELD
10. గడువు ఎలా తీరాలి – timer నా, చదివేటప్పుడా?

Part 5 – పూర్తి system

11. Step – ధర కూడా ఒక మారే నియమమే
12. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

14. నోటీస్ చెప్పాల్సిన English script
15. Follow-ups – distributed locking, waiting list
16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఒక్క వాక్యం – లోపల మూడు సమస్యలు

"Design a movie ticket booking system like BookMyShow."

మనందరికీ ఈ app తెలుసు. అందుకే ప్రమాదం – తెలిసినదాన్ని design చేయడం కష్టం, ఎందుకంటే మనం వాడకందారుగా చూసినదాన్ని model చేయడానికి ప్రయత్నిస్తాం.

ఆగి, లోపల ఉన్న సమస్యలని విడదీద్దాం:

ఒక్క ప్రశ్న, మూడు సమస్యలు • మూడూ వేరే స్వభావం

"Design BookMyShow" – లోపల ఏమి ఉంది

1 • Modelling Movie, Screen, Show, Seat – ఏది దేనికి చెందుతుంది? ఒక కుర్చీ shows మధ్య పంచుకోబడుతుందా? → §4 lo ఇదే విరుగుతుంది	2 • పోటీ ఇద్దరు ఒకే కుర్చీని ఒకే క్షణంలో అడిగితే (Deep Dive 01 §7, ఇక్కడ డబ్బుతో) → §6	3 • Payment ఖాళీ ఎంపిక మరియు చెల్లింపు మధ్య మూడు నిమిషాలు ఉంటాయి ఆ సమయంలో కుర్చీ ఎవరిది? → §8 • ఇదే అసలు problem
--	---	---

ఎవరు ఎక్కడ ఆగిపోతారు

సమస్య 1 ని చాలా మంది గమనించరు – వాళ్ళ design ఒక show కి పనిచేస్తుంది, రెండిటికీ పనిచేయదు
సమస్య 2 ని చాలా మంది గుర్తిస్తారు, కానీ "lock పెడతాను" అని ఆగిపోతారు
సమస్య 3 ని అడిగితేనే చెప్తారు – మేరే లేవనెత్తితే అది ఒక పెద్ద సంతకం
మూడూ కలిపి చెప్పగలిగితే – ఈ interview మీది.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఒక screen lo ఒక రోజు ఎన్ని shows?	ఒకటి కంటే ఎక్కువ అంటే – §4 యొక్క modelling సమస్య తప్పనిసరి
ఎంపిక తర్వాత payment ఎంతసేపు?	ఇదే hold TTL (§9). "వెంటనే" అంటే hold అవసరం లేదు
ఎన్ని seats ఒకసారి book చేయొచ్చు?	1 కంటే ఎక్కువ అంటే all-or-nothing కావాలి (§7)

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Seat రకాలు (silver/gold/recliner)?	ఉంటే ధర seat మీద ఆధారపడుతుంది (§11)
ధర show ని బట్టి మారుతుందా?	Weekend, morning show – ఉంటే pricing ఒక మారే నియమం
ఒకే server నా, పలు servers నా?	ఇది మరే లేవనెత్తాలి – పలు servers అంటే distributed lock (§15)
Cancel చేయొచ్చా? Refund?	Scope. నేను దీన్ని బయట పెడతాను, మరియు అలా చెప్తాను

మొదటి ప్రశ్న అనిపించినంత సాధారణమైనది కాదు

"ఒక screen lo ఒక రోజు ఎన్ని shows?" – జవాబు "నాలుగైదు" అని అందరికీ తెలుసు. కానీ ఆ ప్రశ్న అడగడం వేరే విషయం, ఎందుకంటే అది మీ తర్వాతి అడుగుని నిర్ణయిస్తుంది.

ఒక screen కి ఒకే show అయితే – seat మీద isBooked పెట్టొచ్చు, అది సరైనదే. పలు shows అయితే అది తప్పు, మరియు ఆ తప్పు design lo చాలా లోతుగా కూర్చుంటుంది. §4 lo దాన్ని కళ్ళతో చూద్దాం.

Part 2 – మొదటి విరుపు: ఒక కుర్చీ, రెండు shows

3. Step – అతి సహజమైన మొదటి ప్రయత్నం

Requirement వాక్యంలో nouns ఏమిటి? *Movie, screen, show, seat, booking*. వాటిని classes చేద్దాం. ఒక screen lo seats ఉంటాయి, ఒక seat book అవుతుంది – ఇది సహజంగా అనిపిస్తుంది:

```
class Seat {
  constructor(id) { this.id = id; this.isBooked = false; }
}

class Screen {
  constructor(id, n) {
    this.id = id;
    this.seats = Array.from({ length: n }, (_, i) => new Seat(`A${i + 1}`));
  }
  book(id) {
    const s = this.seats.find((x) => x.id === id);
    if (!s || s.isBooked) return null;
    s.isBooked = true;
    return { screen: this.id, seat: id };
  }
  get free() { return this.seats.filter((s) => !s.isBooked).map((s) => s.id); }
}

class ShowTime {
  constructor(id, movie, screen, time) { Object.assign(this, { id, movie, screen, time }); }
}
```

ఇది **nouns** → **classes** ని అక్షరాలా అనుసరించింది. Seat కి `isBooked` ఉంది. Screen కి seats ఉన్నాయి. Show కి ఒక movie, ఒక screen, ఒక time ఉన్నాయి.

చూడటానికి బాగానే ఉంది. కానీ ఇప్పుడు ఒక నిజమైన theatre ని నడుపుదాం – **ఒక screen, రెండు shows**:

```
const screen1 = new Screen('SCR-1', 3);
const show6pm = new ShowTime('SH-1', 'Bahubali', screen1, '18:00');
const show9pm = new ShowTime('SH-2', 'RRR', screen1, '21:00');

console.log('Asha 6pm shows కి A1 book చేసింది:', show6pm.screen.book('A1'));
console.log('9pm show (వేరే సినిమా) lo ఖాళీ:', show9pm.screen.free);
console.log('Ravi 9pm కి A1 అడిగితే:', show9pm.screen.book('A1'));
```

4. మొదటి విరుపు – 6pm కి book చేస్తే 9pm కూడా పోయింది

Asha 6pm shows కి A1 book చేసింది: { screen: 'SCR-1', seat: 'A1' }
9pm show (వేరే సినిమా) lo ఖాళీ: ['A2', 'A3']
Ravi 9pm కి A1 అడిగితే: null

మొదటి విరుపు – ఇది ఒక **MODELLING** తప్పు, ఒక **BUG** కాదు

Asha **6 గంటల Bahubali** కి A1 తీసుకుంది. ఇప్పుడు **9 గంటల RRR** కి – పూర్తిగా వేరే సినిమా, వేరే సమయం – A1 **అందుబాటులో లేదు**.

Code సరిగ్గానే పనిచేసింది. తప్పు ఏమిటంటే – **isBooked** తప్పు వస్తువు మీద ఉంది.

ఒక కుర్చీ ఒక భౌతిక వస్తువు. అది screen కి చెందుతుంది, మరియు అది ఎప్పుడూ మారదు. కానీ "ఆ కుర్చీ ఖాళీయేనా" అనేది భౌతిక లక్షణం కాదు – అది ఒక నిర్దిష్ట show కి సంబంధించిన స్థితి.

ఇది ఒక పెద్ద, తిరిగి తిరిగి వచ్చే నమూనా

ఈ తప్పుకి ఒక పేరు ఉంది: **identity vs instance** గందరగోళం.

భౌతిక వస్తువు (identity)	ఒక సందర్భంలో దాని స్థితి (instance)
కుర్చీ A1 (screen lo)	6pm show lo A1 booked ; 9pm show lo free
ఒక hotel గది 301	మార్చి 5 కి booked; మార్చి 6 కి free
ఒక విమానం నీటు 12A	ఈ flight lo booked; రేపటి flight lo free
ఒక పుస్తకం (ISBN)	ఈ నిర్దిష్ట కాపీ ఇవ్వబడింది; ఆ కాపీ ఇంకా ఉంది

నియమం: "ఇది ఎప్పుడూ నిజమూ, లేక ఒక నిర్దిష్ట సందర్భంలో మాత్రమేనా?" – సందర్భంలో మాత్రమే అయితే, ఆ field **ఆ సందర్భాన్ని represent చేసే class మీద** ఉండాలి.

5. Step – భౌతిక వస్తువుని, show స్థితిని విడదీయడం

కాబట్టి రెండు classes కావాలి:

```
// భౌతిక కుర్చీ – screen కి చెందినది, ఎప్పుడూ మారదు
class Seat {
  constructor(id, tier) { Object.assign(this, { id, tier }); Object.freeze(this); }
}

// ఒక నిర్దిష్ట show lo ఆ కుర్చీ యొక్క స్థితి
class SeatState {
  #status = 'FREE';
  constructor(seat) { this.seat = seat; }
```

```
// ...  
}
```

Seat ని Object.freeze చేయడం ఒక ఉద్దేశపూర్వక సంకేతం: **ఇది ఎప్పుడూ మారదు**. ఒక కుర్చీ యొక్క id, tier – అవి theatre కట్టినప్పుడు నిర్ణయమై, ఆ తర్వాత మారవు.

ఇప్పుడు Show ప్రతి seat కీ ఒక SeatState ఉంచుకుంటుంది:

```
class Show {  
  #states;  
  constructor(id, movie, screen, startsAt) {  
    Object.assign(this, { id, movie, screen, startsAt });  
    this.#states = new Map(screen.seats.map((s) => [s.id, new SeatState(s)]));  
  }  
}
```

ఒకే screen.seats, కానీ ప్రతి show కీ సొంత #states. 6pm show lo B1 booked అవ్వొచ్చు, 9pm show lo అదే B1 free గా ఉండొచ్చు. §12 lo దీన్ని నడిపి చూస్తాం.

Interview lo ఇది ఒక బలమైన కదలిక. Nouns → classes చేసేటప్పుడు అగి ఇలా చెప్పండి: "Seat is physical and belongs to the screen, but availability belongs to a show. If I put isBooked on the Seat, booking the six o'clock show would block the nine o'clock show. So I'll keep Seat immutable and have each Show own a per-seat state map." – చాలా మంది ఈ తేడాని గమనించరు, మరియు interviewer దాని కోసమే ఎదురుచూస్తుంటాడు.

Part 3 – రెండో విరుపు: ఇద్దరికీ ఒకే కుర్చీ

6. రెండో విరుపు – డబ్బుతో కూడిన race

ఇప్పుడు Show కి ఒక book పేడదాం – "ఖాళీయేనా చూసి, ఆపై ఇవ్వడం":

```
isAvailable(ids) { return ids.every((i) => !this.seats.get(i).isBooked); }

book(ids, user) {
  if (!this.isAvailable(ids)) return null; // అడుగు 1: చూడటం
  ids.forEach((i) => { this.seats.get(i).isBooked = true; }); // అడుగు 2: రాయడం
  return { showId: this.id, seats: ids, user };
}
```

చివరి ఒక్క కుర్చీ మిగిలింది, ఇద్దరు users ఒకేసారి అడిగారు. JavaScript single-threaded కాబట్టి Java threads చేసే పనిని **చేతితో అదే క్షణంలో** నడుపుదాం:

```
const show = new Show('S1', ['A1']); // చివరి ఒక్క seat
const okAsha = show.isAvailable(['A1']); // Asha: "A1 ఖాళీ"
const okRavi = show.isAvailable(['A1']); // Ravi: "A1 ఖాళీ" ← Asha ఇంకా
రాయలేదు
// ... ఇద్దరూ book చేస్తారు ...
```

```
Asha ticket: { seats: [ 'A1' ], user: 'asha' }
Ravi ticket: { seats: [ 'A1' ], user: 'ravi' }
=> ఇద్దరికీ A1 ఇచ్చాం. ఇద్దరూ డబ్బు కట్టారు. theatre lo ఒకే కుర్చీ.
```

రెండో విరుపు – DEEP DIVE 01 §7, కానీ ఇప్పుడు డబ్బుతో

ఇది సరిగ్గా అదే **check-then-act race** – Parking Lot lo రెండు gates ఒకే spot ని పట్టుకున్నట్టు. కానీ ఇక్కడ పరిణామం చాలా ఘోరం: **ఇద్దరూ డబ్బు కట్టారు**. ఇద్దరూ theatre కి వచ్చారు. ఒకే కుర్చీ.

Parking lot lo ఇది ఒక చిరాకు. ఇక్కడ ఇది ఒక **refund**, ఒక ఫిర్యాదు, మరియు ఒక చెడ్డ review.

7. Step – Atomic claim, మరియు కొత్త కష్టం: అన్నీ లేదా ఏమీ లేదు

పరిష్కారం తెలుసు – చూడటం + రాయడం ఒకే విడదీయలేని ఆపరేషన్, మరియు అది **seat మీదే**:

```
class Seat {
  #bookedBy = null;
```

```

constructor(id) { this.id = id; }
get isFree() { return this.#bookedBy === null; }
tryClaim(user) { if (this.#bookedBy !== null) return false; this.#bookedBy = user;
return true; }
release() { this.#bookedBy = null; }
}

```

కానీ ఇక్కడ Parking Lot లో లేని **ఒక కొత్త కష్టం** ఉంది.

Parking lot లో ఒక car కి **ఒకే spot** కావాలి. ఇక్కడ ఒక కుటుంబానికి **నాలుగు కుర్చీలు కలిసి** కావాలి. మరి మూడు దొరికి, నాలుగోది దొరకకపోతే?

ఆ మూడింటినీ తిరిగి వదిలేయాలి. లేకపోతే అవి ఎవరికీ ఉపయోగపడకుండా ఇరుక్కుపోతాయి – user కి ticket రాలేదు, కానీ కుర్చీలు పోయాయి.

```

book(ids, user) {
  const claimed = [];
  for (const i of ids) {
    const seat = this.seats.get(i);
    if (!seat || !seat.tryClaim(user)) {
      claimed.forEach((s) => s.release()); // ← అన్నీ దొరకకపోతే వెనక్కి
      return null;
    }
    claimed.push(seat);
  }
  return { showId: this.id, seats: ids, user };
}

```

నడిపి చూద్దాం – Asha A1, A2 తీసుకుంది; Ravi A2, A3 అడిగాడు (A2 పోయింది):

```

Asha A1,A2 → { showId: 'S1', seats: [ 'A1', 'A2' ], user: 'asha' }
Ravi A2,A3 → null
rollback తర్వాత ఖాళీ: [ 'A3' ] ← A3 తిరిగి వచ్చింది
Ravi A3 → { showId: 'S1', seats: [ 'A3' ], user: 'ravi' }

```

మూడో పంక్తి ముఖ్యమైనది. Ravi యొక్క ప్రయత్నం విఫలమైంది, కానీ అతను ఇప్పటికే పట్టుకున్న **A3 తిరిగి ఖాళీ అయింది** – కాబట్టి అతను దాన్ని ఒక్కడాన్నే తీసుకోగలిగాడు.

ఇదే atomicity యొక్క అర్థం – "అన్నీ జరుగుతాయి, లేదా ఏమీ జరగదు". Database transaction కూడా చేసేది ఇదే. ఇక్కడ మనం చేతితో చేస్తున్నాం ఎందుకంటే ఇది in-memory. **Interview లో ఈ rollback ని చూపించడం** – ఇది చాలా మంది మర్చిపోయే వివరం, మరియు ఇది మీరు multi-seat booking గురించి నిజంగా ఆలోచించారని చూపిస్తుంది.

Part 4 – మూడో విరుపు: payment ఖాళీ

8. మూడో విరుపు – రెండు దారులు, రెండూ తప్పు

ఇప్పుడు మన `book()` సరిగ్గా పనిచేస్తోంది. కానీ ఒక నిజమైన వాడకందారుని అనుసరిద్దాం:

1. Asha seats ఎంచుకుంది – **10:00:00**
2. Payment page తెరిచింది – **10:00:05**
3. OTP కోసం ఎదురుచూసింది, UPI app తెరిచింది... – **10:02:30**
4. Payment విజయవంతం – **10:03:10**

ఆ మూడు నిమిషాల్లో ఆ కుర్చీలు ఎవరివి?

ఈ ప్రశ్నకి రెండు సహజమైన జవాబులు ఉన్నాయి, మరియు **రెండూ తప్పు**:

దృశ్యం A – ఎంపిక వద్దే `book` చేస్తే
Asha ఎంచుకుంది. ఖాళీ: []
...Asha payment page తెరిచి, browser మూసేసింది. డబ్బు కట్టలేదు.
ఒక గంట తర్వాత ఖాళీ: [] ← ఇంకా ఎవరికీ దొరకవు

దృశ్యం B – payment తర్వాతే `book` చేస్తే
Asha A1 ఎంచుకుంది (claim లేదు). Ravi కూడా A1 ఎంచుకున్నాడు.
ఇద్దరూ payment page కి వెళ్ళారు. ఇద్దరూ ₹250 కట్టారు.
confirm: Asha → ticket
confirm: Ravi → విఫలం ← డబ్బు తీసుకుని refund చేయాలి

మూడో విరుపు – ఇదే ఈ PROBLEM యొక్క నిజమైన గుండె

దృశ్యం A: ఎంపిక వద్దే పక్కాగా `book` చేస్తే – payment చేయని ప్రతి వ్యక్తీ ఆ కుర్చీలని **శాశ్వతంగా** ఆక్రమిస్తాడు. ఒక blockbuster విడుదల రోజున, browse చేసి వెళ్ళిపోయే వందల మంది మొత్తం theatre ని నింపేస్తారు – ఒక్క ticket కూడా అమ్ముడుపోకుండా.

దృశ్యం B: payment తర్వాతే `book` చేస్తే – ఇద్దరూ డబ్బు కట్టాక అప్పుడు తెలుస్తుంది ఒకరికి కుర్చీ లేదని. **Refund, క్షమాపణ, కోపం.**

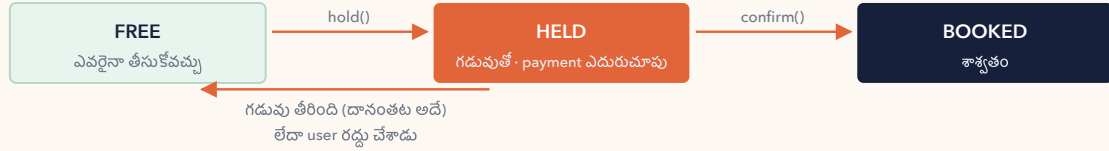
అంటే మనకి కావలసినది ఒక **మూడో స్థితి** – "ఇంకా ఎవరిదీ కాదు, కానీ ఇప్పుడే ఎవరికీ ఇవ్వకూడదు". **తాత్కాలికం**, మరియు అది **దానంతట అదే వదిలిపోవాలి**.

9. Step – మూడో స్థితి: HELD

ఒక కుర్చీకి ఇప్పుడు **రెండు కాదు, మూడు స్థితులు**:

ఒక కుర్చీ యొక్క జీవితచక్రం - మూడు స్థితులు, ఐదు మార్పులు

SEAT STATE MACHINE - ప్రతి మార్పుకీ ఒక కారణం



ఇక్కడ మూడు విషయాలు గమనించండి

1. **BOOKED నుంచి బయటికి బాణం లేదు** - ఈ scope lo cancel లేదు. ఉంటే అది ఒక కొత్త మార్పు.
 2. **HELD → FREE అనేది ఎవరూ పీలవని మార్పు** - సమయం దాన్ని జరిపిస్తుంది (§10)
 3. **FREE → BOOKED అనే నేరుగా దారి లేదు** - ప్రతి booking hold గుండా వెళ్ళాలి
- ఆ మూడే అంశమే §8 lo చూసిన రెండు దృశ్యాలనీ అసాధ్యం చేస్తుంది.

```
const Status = Object.freeze({ FREE: 'FREE', HELD: 'HELD', BOOKED: 'BOOKED' });

class SeatState {
  #status = Status.FREE; #holder = null; #until = 0;
  constructor(seat) { this.seat = seat; }

  // గడువు తీరిన hold ని *చదివేటప్పుడే* వదిలేయడం
  #settle(now) {
    if (this.#status === Status.HELD && now >= this.#until) {
      this.#status = Status.FREE; this.#holder = null; this.#until = 0;
    }
  }

  statusAt(now) { this.#settle(now); return this.#status; }

  tryHold(user, now, ttl) {
    this.#settle(now);
    if (this.#status !== Status.FREE) return false;
    this.#status = Status.HELD; this.#holder = user; this.#until = now + ttl;
    return true;
  }

  confirm(user, now) {
    this.#settle(now);
    if (this.#status !== Status.HELD || this.#holder !== user) return false; // ←
    // holder check
    this.#status = Status.BOOKED; this.#until = 0;
    return true;
  }

  release(user) {
    if (this.#status === Status.HELD && this.#holder === user) {
      this.#status = Status.FREE; this.#holder = null; this.#until = 0;
    }
  }
}
```

#holder తనిఖీ ఎందుకు అంత ముఖ్యం

confirm lo `this.#holder !== user` అనే షరత గమనించండి. ఇది లేకపోతే - **Asha hold చేసిన కుర్చీని Ravi confirm చేయగలడు.** release lo కూడా అదే: మీరు పట్టుకోని కుర్చీని వదిలేయలేరు.

ఇవి చిన్న షరతులు, కానీ ఇవే **ownership** ని అమలు చేస్తాయి. ఒక booking system lo ఇది భద్రతా విషయం, కేవలం ఒక correctness విషయం కాదు.

10. గడువు ఎలా తీరాలి – timer నా, చదివేటప్పుడా?

Hold 5 నిమిషాల్లో గడువు తీరాలి. దాన్ని ఎలా అమలు చేయాలి?

దారి 1 – ప్రతి hold కీ ఒక timer. `setTimeout(() => seat.release(), 300_000)`.

సరళంగా అనిపిస్తుంది, కానీ: ఒక blockbuster విడుదల రోజున ఒకేసారి **లక్షల holds** ఉంటాయి. అంటే లక్షల timers. మరియు user ముందే confirm చేస్తే ఆ timer ని రద్దు చేయాలి – లేకపోతే అది తర్వాత వచ్చి ఒక **booked** కుర్చీని విడిపించేస్తుంది.

దారి 2 – చదివేటప్పుడు తేల్చడం (lazy). ఎవరైనా ఆ కుర్చీని చూసినప్పుడు – "ఈ hold గడువు తీరిందా?" అని అప్పుడు లెక్కించడం. అదే పైన `#settle(now)`.

	Timer	Lazy (#settle)
లక్ష holds కి ఖర్చు	లక్ష timers	సున్నా
Confirm తర్వాత శుభ్రపరచడం	Timer రద్దు చేయాలి (మర్చిపోతే bug)	అవసరం లేదు
Memory	గడువు తీరిన hold వెంటనే పోతుంది	ఎవరూ చూడకపోతే కూర్చీని ఉంటుంది
ఖచ్చితత్వం	సరిగ్గా 5:00 కి	చదివినప్పుడు సరైనది – అదే మనకి కావలసినది

Lazy గెలిచింది, ఎందుకంటే – ఒక గడువు తీరిన hold ఎవరికీ కనిపించనంత కాలం, అది ఎవరికీ హాని చేయదు. ఎవరైనా ఆ కుర్చీని అడిగిన క్షణం, అది ఖాళీగా కనిపిస్తుంది. అదే సరైన ప్రవర్తన.

ఇది Deep Dive 02 §15 lo చూసిన అదే నమూనా – cache TTL కూడా lazy expiry వాడుతుంది, సరిగ్గా అదే కారణాలకి. **మూడు docs lo మూడేసారి:** సమయాన్ని ఒక `clock` గా బయట నుంచి ఇవ్వడం, మరియు గడువుని చదివేటప్పుడు తేల్చడం.

ఒక ఆచరణాత్మక జోడింపు: memory సమస్య నిజమే, కాబట్టి production lo ఒక నెమ్మదైన **sweeper** కూడా ఉంటుంది – ప్రతి కొన్ని నిమిషాలకీ పాత holds ని తుడిచేది. కానీ అది ఒక శుభ్రత పని, ఒక **correctness** పని కాదు. ఆ తేడాని interview lo చెప్పడం మంచిది.

Hold జీవితచక్రాన్ని నడిపి చూద్దాం:

```
Asha hold → [ 'A1', 'A2' ] • గడువు 300 సెకన్ల వద్ద
ఇప్పుడు ఖాళీ: []
Ravi A1 అడిగితే: null
2 నిమిషాల తర్వాత ఖాళీ: [] (hold ఇంకా సజీవం)
```

5 నిమిషాల తర్వాత ఖాళీ: ['A1', 'A2'] ← hold దానంతట అదే పోయింది
Ravi ఇప్పుడు అడిగితే: దొరికింది ✓

ఐదో పంక్తి ముఖ్యమైనది. ఎవరూ ఏమీ పిలవలేదు, ఏ timer నడవలేదు – కానీ కుర్చీలు తిరిగి ఖాళీ అయ్యాయి.
§8 యొక్క దృశ్యం A పరిష్కారమైంది.

Part 5 – పూర్తి system

11. Step – ధర కూడా ఒక మారే నియమమే

ఒక కుర్చీ ధర ఎంత? ఇది ఒక సంఖ్య కాదు – ఇది ఒక **నియమం**, మరియు అది తరచుగా మారుతుంది:

- Seat tier ని బట్టి – silver ₹150, gold ₹250, recliner ₹450
- Weekend lo ఎక్కువ
- Morning show lo తక్కువ
- ప్రతి ticket కి ఒక convenience fee
- Coupon, offer, loyalty points...

Deep Dive 01 §12 lo చూసిన అదే ఆకారం: **నియమాలు ఒకదాన్ని ఒకటి మారుస్తాయి, భర్తీ చేయవు.** కాబట్టి మళ్ళీ **Decorator** పొరలు:

```
class TierPricing {
  constructor(rates) { this.rates = rates; }
  price(seat) { return this.rates[seat.tier]; }
}
class WeekendSurcharge {
  constructor(inner, factor = 1.25) { Object.assign(this, { inner, factor }); }
  price(seat, ctx) { return this.inner.price(seat, ctx) * (ctx.isWeekend ?
this.factor : 1); }
}
class ConvenienceFee {
  constructor(inner, per = 30) { Object.assign(this, { inner, per }); }
  price(seat, ctx) { return this.inner.price(seat, ctx) + this.per; }
}
```

వాటిని పేర్పడం:

```
const pricing = new ConvenienceFee(new WeekendSurcharge(
  new TierPricing({ SILVER: 150, GOLD: 250, RECLINER: 450 })));
```

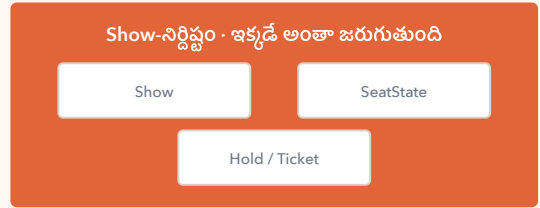
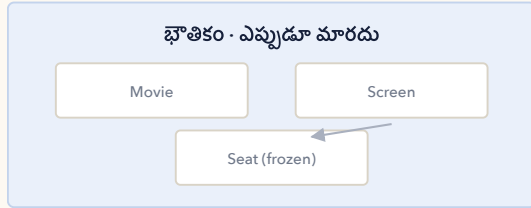
పేర్లు క్రమం ముఖ్యం, మరియు ఇది ఒక business నిర్ణయం. ఇక్కడ convenience fee బయటిపొర, అంటే అది weekend surcharge తర్వాత కలుస్తుంది – కాబట్టి fee మీద surcharge పడదు. తిప్పితే fee కూడా 1.25 రెట్లు అవుతుంది.

Interview lo ఈ ప్రశ్నని మరే లేవనెత్తండి: "Should the convenience fee be taxed by the weekend surcharge or not? That's the order of the wrappers, and it's a product decision, not a technical one." – ఇది మీరు code ని business తో కలిపి ఆలోచిస్తున్నారని చూపిస్తుంది.

12. మొత్తం code ఒకే చోట • నడిపి చూద్దాం

నిర్మాణం • భౌతికం ఎడమ వైపు, SHOW-నిర్దిష్టం కుడి వైపు

ఈ నిలువు గీతే 54 మొక్క పాఠం



ఒకే Screen, ఒకే Seat objects – కానీ ప్రతి Show కీ సొంత SeatState map

మారే నియమాలు – రెండూ బయట

Pricing – TierPricing + WeekendSurcharge + ConvenienceFee (పొరలు)

clock – hold గడువు లెక్కించడానికి; test lo నకిలీ గడియారం

Show కి ఈ రెండింటి **లోపలి నియమం** తెలియదు – కేవలం ఒప్పందం తెలుసు

```
const Tier = Object.freeze({ SILVER: 'SILVER', GOLD: 'GOLD', RECLINER: 'RECLINER' });
const Status = Object.freeze({ FREE: 'FREE', HELD: 'HELD', BOOKED: 'BOOKED' });

class Seat {
  constructor(id, tier) { Object.assign(this, { id, tier }); Object.freeze(this); }
}

class Screen { constructor(id, seats) { Object.assign(this, { id, seats }); } }
class Movie { constructor(id, title, runtimeMin) { Object.assign(this, { id, title, runtimeMin }); } }

class SeatState {
  #status = Status.FREE; #holder = null; #until = 0;
  constructor(seat) { this.seat = seat; }
  #settle(now) {
    if (this.#status === Status.HELD && now >= this.#until) {
      this.#status = Status.FREE; this.#holder = null; this.#until = 0;
    }
  }
  statusAt(now) { this.#settle(now); return this.#status; }
  tryHold(user, now, ttl) {
    this.#settle(now);
    if (this.#status !== Status.FREE) return false;
    this.#status = Status.HELD; this.#holder = user; this.#until = now + ttl;
    return true;
  }
  confirm(user, now) {
    this.#settle(now);
    if (this.#status !== Status.HELD || this.#holder !== user) return false;
    this.#status = Status.BOOKED; this.#until = 0; return true;
  }
  release(user) {
    if (this.#status === Status.HELD && this.#holder === user) {
      this.#status = Status.FREE; this.#holder = null; this.#until = 0;
    }
  }
}

class TierPricing {
```

```

    constructor(rates) { this.rates = rates; }
    price(seat) { return this.rates[seat.tier]; }
  }
  class WeekendSurcharge {
    constructor(inner, factor = 1.25) { Object.assign(this, { inner, factor }); }
    price(seat, ctx) { return this.inner.price(seat, ctx) * (ctx.isWeekend ?
this.factor : 1); }
  }
  class ConvenienceFee {
    constructor(inner, per = 30) { Object.assign(this, { inner, per }); }
    price(seat, ctx) { return this.inner.price(seat, ctx) + this.per; }
  }

  class Show {
    #states;
    constructor(id, movie, screen, startsAt, { holdMs = 300_000, clock = () =>
Date.now(),
                                pricing, isWeekend = false } = {}) {
      Object.assign(this, { id, movie, screen, startsAt, holdMs, clock, pricing,
isWeekend });
      this.#states = new Map(screen.seats.map((s) => [s.id, new SeatState(s)]));
    }

    availableSeats() {
      const now = this.clock();
      return [...this.#states.values()]
        .filter((st) => st.statusAt(now) === Status.FREE)
        .map((st) => ({ id: st.seat.id, tier: st.seat.tier,
                        price: this.pricing.price(st.seat, { isWeekend: this.isWeekend
}) }));
    }

    hold(seatIds, user) {
      const now = this.clock(), taken = [];
      for (const id of seatIds) {
        const st = this.#states.get(id);
        if (!st || !st.tryHold(user, now, this.holdMs)) {
          taken.forEach((x) => x.release(user)); // అన్నీ లేదా ఏమీ లేదు
          return { ok: false, reason: `SEAT_UNAVAILABLE: ${id}` };
        }
        taken.push(st);
      }
      const total = seatIds.reduce((sum, id) =>
sum + this.pricing.price(this.#states.get(id).seat, { isWeekend: this.isWeekend
}), 0);
      return { ok: true, hold: { holdId: `H-${this.id}-${user}-${now}`, showId:
this.id,
                                seats: seatIds, user, total, expiresAt: now +
this.holdMs } };
    }

    confirm(hold) {
      const now = this.clock();
      const bad = hold.seats.find((id) => this.#states.get(id).statusAt(now) !==
Status.HELD);
      if (bad) return { ok: false, reason: `HOLD_EXPIRED: ${bad}` };
      hold.seats.forEach((id) => this.#states.get(id).confirm(hold.user, now));
      return { ok: true, ticket: { ticketId: `T-${hold.holdId}`, showId: this.id,
                                movie: this.movie.title, seats: hold.seats,

```

```

        user: hold.user, paid: hold.total } });
    }

    cancelHold(hold) { hold.seats.forEach((id) =>
    this.#states.get(id).release(hold.user)); }
}

```

ఇప్పుడు **ఒకే screen, రెండు shows** – §4 యొక్క విరుపుని పరీక్షిద్దాం:

```

const seats = [new Seat('A1', Tier.SILVER), new Seat('A2', Tier.SILVER),
    new Seat('B1', Tier.GOLD), new Seat('B2', Tier.GOLD),
    new Seat('C1', Tier.RECLINER)];
const screen = new Screen('SCR-1', seats);
const bahubali = new Movie('M1', 'Bahubali', 159);
const rrr = new Movie('M2', 'RRR', 187);
const pricing = new ConvenienceFee(new WeekendSurcharge(
    new TierPricing({ SILVER: 150, GOLD: 250, RECLINER: 450 })));

let now = 0;
const show6 = new Show('SH-6PM', bahubali, screen, '18:00',
    { clock: () => now, pricing, isWeekend: true });
const show9 = new Show('SH-9PM', rrr, screen, '21:00',
    { clock: () => now, pricing, isWeekend: true });

```

6pm ధరలు: A1/SILVER=₹217.5 A2/SILVER=₹217.5 B1/GOLD=₹342.5 B2/GOLD=₹342.5
C1/RECLINER=₹592.5

Asha hold → true | seats ['B1', 'B2'] | మొత్తం ₹685

6pm ఖాళీ : A1, A2, C1

9pm ఖాళీ : A1, A2, B1, B2, C1 ← B1, B2 ఇక్కడ ఇంకా ఖాళీ

Ravi B1 6pm: SEAT_UNAVAILABLE: B1

Ravi B1 9pm: దొరికింది ✓

Asha payment పూర్తి (100 సెకన్లలో)

```

confirm → {
  ticketId: 'T-H-SH-6PM-asha-0',
  showId: 'SH-6PM',
  movie: 'Bahubali',
  seats: [ 'B1', 'B2' ],
  user: 'asha',
  paid: 685
}

```

Meera అలస్యంగా కట్టింది (400 సెకన్లు)

```

confirm → { ok: false, reason: 'HOLD_EXPIRED: A1' }
A1 ఇప్పుడు: ఖాళీ ✓

```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

ధరలు. SILVER = $150 \times 1.25 + 30 = \mathbf{217.5}$. GOLD = $250 \times 1.25 + 30 = \mathbf{342.5}$. RECLINER = $450 \times 1.25 + 30 = \mathbf{592.5}$. మూడు పౌరలూ సరైన క్రమంలో పనిచేశాయి.

మూడు, నాలుగో పంక్తులు – §4 యొక్క పరిష్కారం. B1, B2 6pm show lo hold అయ్యాయి, కానీ **9pm show lo ఇంకా ఖాళీగానే ఉన్నాయి**. ఇది సరిగ్గా అదే `Seat` object, కానీ వేరే `SeatState`. మొదటి విరుపు పూర్తిగా పరిష్కారమైంది.

Ravi B1 6pm: SEAT_UNAVAILABLE: B1. తిరస్కరణ ఒక bare `null` కాదు – **ఏ seat వల్ల విఫలమైందో** చెప్తోంది. UI ఆ నిర్దిష్ట కుర్చీని ఎరుపు చేసి "ఇది ఇప్పుడే ఎవరో తీసుకున్నారు" అని చూపించగలదు.

paid: 685. రెండు GOLD కుర్చీలు = 342.5×2 . Hold సమయంలో లెక్కించిన ధరే ticket మీద ఉంది – **payment తర్వాత ధర మారదు**. (Hold సమయంలో ధరని స్థిరం చేయడం ఒక ఉద్దేశపూర్వక నిర్ణయం.)

చివరి రెండు పంక్తులు – §8 యొక్క పరిష్కారం. Meera 400 సెకన్లు తీసుకుంది, hold గడువు 300. ఆమె confirm **విఫలమైంది**, మరియు A1 **ఖాళీగా తిరిగి వచ్చింది**. ఏ timer నడవలేదు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>Movie</code> , <code>Screen</code> , <code>Seat</code> , <code>Show</code>	Nouns → classes
§4 (విరుపు)	<code>SeatState</code> (show వారీగా)	ఒక కుర్చీ shows మధ్య పంచుకోబడుతుంది
§6 (విరుపు)	<code>tryHold</code> – చూడటం+రాయడం కలిపి	Check-then-act race
§7	Rollback (<code>taken.forEach(release)</code>)	పలు seats – అన్నీ లేదా ఏమీ లేదు
§8 (విరుపు)	<code>HELD</code> స్థితి + <code>expiresAt</code>	Payment ఖాళీ
§10	<code>#settle(now)</code> – lazy expiry	లక్షల timers వద్దు
§11	Pricing పోరలు	ధర ఒక మారే business నియమం

Part 6 – Interview Io

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

ఈ problem io modelling మీద ఎక్కువ సమయం పెట్టాలి – అదే వేరుపాటు

5 నిమి

Clarify – "ఒక screen io ఎన్ని shows?" మర్చిపోవద్దు

10 నిమి – Modelling

Seat vs SeatState - ఇదే మీ మొదటి ముద్ర

14 నిమి – Hold lifecycle

మూడు states + rollback + lazy expiry

7 నిమి

Pricing పొరలు (రెండు రాసి చాలు)

9 నిమి

Concurrency + distributed + follow-ups

ఏమి తప్పక చెప్పాలి

1. **Seat vs SeatState** (§4-5) – "availability belongs to the show, not the seat." ఇది మీ మొదటి పెద్ద ముద్ర.
2. **Multi-seat rollback** (§7) – "మూడు దొరికి నాలుగోది దొరకకపోతే మూడింటినీ వదిలేస్తాను".
3. **Payment ఖాళీ, మరియు HELD స్థితి** (§8-9) – **మరే లేవనెత్తండి.** ఇదే ఈ problem యొక్క గుండె.
4. **Lazy expiry** (§10) – timers ఎందుకు వద్దో.

ఏమి వదిలేయాలి

- మూడు pricing decorators రాయొడ్లు – రెండు రాసి "మిగతావి ఇదే నమూనా" అనండి.
- `availableSeats()` ని రాయొడ్లు – అది స్పష్టం.
- **Cancel/refund** – scope బయట అని మీరే చెప్పండి.

14. నీటితో చెప్పాల్సిన English script

"A few questions first. How many shows run on one screen in a day? How long does a user get between selecting seats and paying? And can they book multiple seats at once?"

I'll start with modelling, because there's a trap here. A seat is physical – it belongs to a screen and never changes. But *whether it's available* isn't a property of the seat, it's a property of a **show**. If I put `isBooked` on the `Seat`, then booking the six o'clock show also blocks the nine o'clock show on the same screen, which is a different movie. So `Seat` stays immutable, and each `Show` owns a map of per-seat state. This is the same identity-versus-instance split you'd need for hotel rooms or flight seats.

For booking, the naive version checks availability then writes, and that's a check-then-act race – two users can both see the last seat as free. So the claim goes on the seat state itself, as one operation that returns a boolean.

There's an extra wrinkle here that a parking lot doesn't have: people book several seats together. If three succeed and the fourth is gone, I have to release the three, otherwise they're stranded – the user got no ticket but the seats are held. So it's all-or-nothing with rollback.

Now the part I think is the real problem. There's a gap of two or three minutes between choosing seats and completing payment. If I book at selection time, anyone who abandons the page holds those seats forever – on a big release that fills the theatre with nothing sold. If I book only after payment, two people can both pay for the same seat and one gets a refund and a bad experience.

So a seat needs three states, not two: free, **held**, and booked. A hold has an owner and an expiry. Confirm only works if you're the holder and the hold is still alive.

For expiry I wouldn't use a timer per hold – on a blockbuster that's hundreds of thousands of timers, and you'd have to cancel each one on confirm or it comes back later and frees a booked seat. Instead I settle lazily: when anyone reads the seat's status, I check whether the hold has expired. An expired hold that nobody has looked at yet can't hurt anyone. I'd still add a slow background sweeper, but that's housekeeping, not correctness.

Pricing I'd keep outside the `Show` – tier, weekend surcharge, convenience fee – as wrappers, because those change constantly and shouldn't mean editing the booking logic. One product question there: should the convenience fee be inside or outside the weekend multiplier? That's the wrapper order.

One weakness: my `availableSeats` scans every seat. For a home page showing counts across hundreds of shows I'd keep a running free-count per show instead."

15. Follow-ups – distributed locking, waiting list

Follow-up	జవాబు	మారే classes
"Seat tiers కి వేరే ధర"	ఇప్పటికే ఉంది (§11)	0
"Morning show discount"	కొత్త <code>TimeOfDayDiscount</code> పొర	+1 కొత్తది
"Coupon codes"	కొత్త పొర, <code>ctx</code> lo coupon	+1 కొత్తది
"Hold సమయం VIP users కి ఎక్కువ"	<code>holdMs</code> ని user tier నుంచి తీసుకోవడం	<code>hold()</code> lo ఒక parameter
"ఒక్కరు 10 కంటే ఎక్కువ tickets కొనకూడదు"	<code>hold()</code> lo ఒక తనిఖీ, లేదా ఒక policy object	Show
"Seats కలిసి ఉండాలి (వేరువేరుగా వద్దు)"	ఒక <code>SeatLayout</code> + adjacency తనిఖీ	+1 కొత్తది
"50 servers"	కింద చూడండి	Seat claim యొక్క అమలు
"Sold out అయితే waiting list"	కింద చూడండి	+1 కొత్తది

50 servers అయితే

"అప్పుడు *in-memory* `SeatState` పనిచేయదు – ప్రతి server కి వేరే copy ఉంటుంది. State ఒక చోట ఉండాలి.

నేను DB conditional update ఎంచుకుంటాను, ఒక distributed lock కాదు:

```
UPDATE seat_state SET status='HELD', holder=?, hold_until=?
WHERE show_id=? AND seat_id=? AND (status='FREE' OR hold_until < NOW())
```

ఇది **ఒక్క atomic ఆపరేషన్**. Affected rows 1 అయితే మీరు గెలిచారు, 0 అయితే ఇంకెవరో ముందు తీసుకున్నారు. ఇది Deep Dive 01 §9 lo చూసిన అదే నమూనా.

ఎందుకు distributed lock కాదు: Redis lock lo lock పట్టుకున్న server పడిపోతే ఆ కుర్చీ lock TTL వరకు ఇరుక్కుపోతుంది. Conditional update lo ఆ సమస్య లేదు – నిజం ఎప్పుడూ DB row lo ఉంటుంది, ఒక వేరే చోట ఉన్న lock lo కాదు.

గమనించండి – `hold_until < NOW()` అనే ఆ షరత్ మన lazy expiry నే SQL lo రాసింది. **ఒకే ఆలోచన, రెండు చోట్లు.**"

Waiting list

"Sold out అయ్యాక, users ని ఒక queue lo చేర్చిచ్చు. ఎవరైనా cancel చేస్తే, queue lo మొదటివారికి ఒక పరిమిత-కాల hold ఇవ్వడం – సరిగ్గా మన **HELD** స్థితే, కానీ ఈసారి system ఇచ్చినది, user అడిగినది కాదు.

అంటే **waiting list కి కొత్త state machine అవసరం లేదు** – ఉన్నదే సరిపోతుంది. అది ఒక మంచి సంకేతం: మన abstraction సరైన చోట ఉంది."

16. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకొక్కడ వస్తుంది
Identity vs instance	Seat vs SeatState (§4)	Hotel room, flight seat, book copy, product vs SKU
Check-then-act race	ఇద్దరికీ ఒకే కుర్చీ (§6)	Parking lot, inventory, wallet, LRU
All-or-nothing + rollback	పలు seats (§7)	Transactions, saga, batch operations
తాత్కాలిక hold	HELD + TTL (§9)	Flight/hotel booking, cart reservation, OTP
Lazy expiry	#settle(now) (§10)	Cache TTL, session, token
మారే నియమాన్ని బయట	Pricing పొరలు (§11)	Discounts, fees, rate limits
Conditional update	WHERE status='FREE' (§15)	ఏ distributed claim lo అయినా

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 LRU & LFU Cache · 03 Rate Limiter · 04 BookMyShow · 06 Elevator.**

తర్వాతిది: **05 Splitwise.**

ఒక నమూనా గమనించండి: check-then-act race ఇప్పుడు **రెండు** docs lo వచ్చింది (01, 04), మరియు lazy expiry కూడా **రెండిట్లో** (02, 04). ఇవి యాదృచ్ఛికం కాదు – LLD interviews ఒకే కొన్ని ఆలోచనలని వేర్వేరు domains lo తిరిగి అడుగుతాయి. అందుకే ఒక problem ని లోతుగా చేస్తే, తర్వాతిది సగం సులభమవుతుంది.

వేగవంతమైన revision కోసం – **LLD_Design_Problems_Telugu.pdf** lo Problem 03.

BookMyShow – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output నిజంగా **node** lo run చేసి తీసినదే ✓